Introductory Words:

This project started very well for me: Before coming to CGL I studied math for a few semesters. Therefore, I was very happy to hear that a client asked for an educational math game.
In my enthusiasm I took the project lead role, next to my design role; Additionally, due to only having one programmer, I also helped out with the programming of the project.
Therefore this post-portem is separated into three sections:

- Project Management
- Design
- Programming

Project Lead/Management

Task 1: Distributing Workload

Problem 1.A: programming bottleneck

With three designers, but only one programmer, we had a programming bottleneck; Additionally early tries to distribute some of Ahsans coding work to Frieda and me resulted in conflicting code and therefore more issues.

Solution 1.A: Seperating Code parts

However, having run into the code conflicts lead to a good conversation about how we can split the coding a bit more effectively. We figured out some areas that are easily separatable:

- Ahsan does the coding for all the game's main functionality
  (fraction engine, card interactions, enemies, rewards, ...)
- Visualising fraction as figures becomes my task
- Frieda implements UI/UX as far as possible and documents the rest for Ahsan to implement
- Randomisation of enemies and cards (Level Design implementation) becomes Vin's task

With that our programming bottleneck was mostly (as expected during team-forming phase) solved.

Problem 1.B.i: finding work for artists

During group-forming we had no artists at first. At that point we also didn't know what our game would be, but given the clients kick-off pitch it seemed to be strongly UI based.
As some drama started, I suggested that we could first talk with our client, what type of game we'd make, before pitching that to artists so that some of them join our group. However, while this suggestion was acknowledged, it got completely ignored. Thus two artists were pressured into our group and we had to make sure that there is a valuable contribution they can make to the game.

Solution 1.B: Turning 2D to 3D

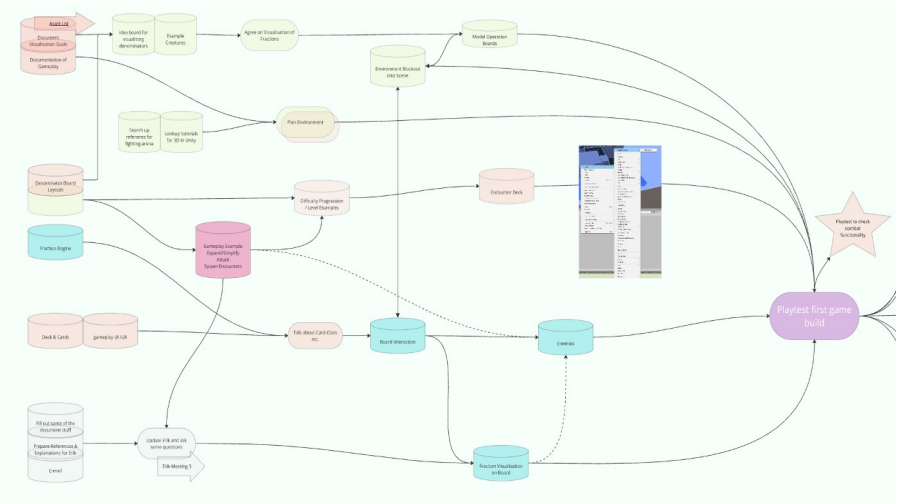
Consequently our group figured out a way to adjust the game so that artists can contribute something: Luckily Carly wanted to do Environment while Oliver tended towards creature modelling. Therefore we adjusted the UI-elements of our game to be in a 3D-Environment. Additionally the typically simplified 2D-visualisation of fractions from math-books was turned into a 3D-figure-based visualisation.

Conclusion 1.A & 1.B:

The above work distribution wroked well to ensure everybody in the group contributed something important to make the game better. However this puts the work distribution over a good game concept itself. In my opinion it would be better to first have an idea of the contents & required assets for a game, before forming groups.

Problem 1.C: documenting who does which tasks

Throughout the project I tried a visual system for showing how work is distributed and which tasks are dependend on another. This turned out to grow out of proportion, because more tasks quickly turned into a complicated net.

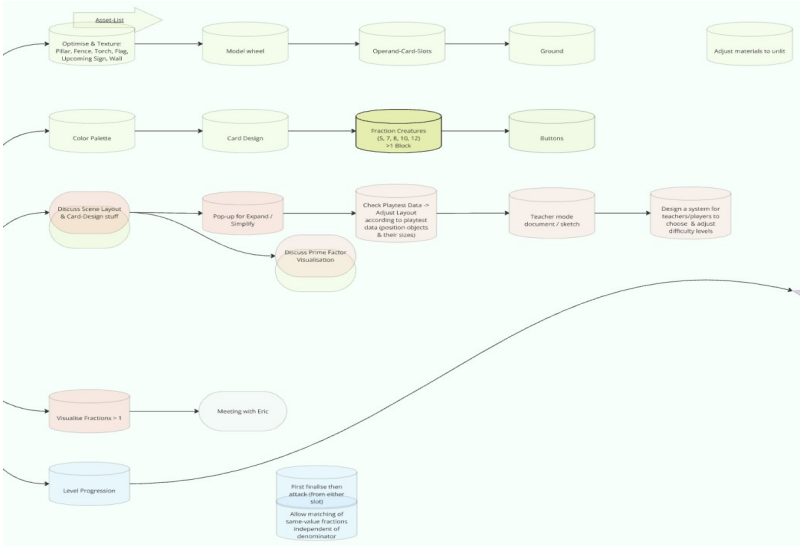


Learning 1.C:

Additionally I learned that a rough overview is enough for most people. What people really need is a task-list for themselves to know what they've got left to work on.

Solution 1.C:

At first I changed the system to use one track per person. This however removed the reason to use a visual system, thus towards the end of the project (and for upcoming projects), I'll go back to using todo-lists.



Unsolved Problem 1.D: crunching instead of planning

At the end of the project, we still had a lot of tasks left (creating a tutorial, fixing bugs, connecting the main-menu with levels, …). Overwhelmed by this, I only focused on solving as many of these issues as possible without considering which could still be done in time and how to distribute task. As a result I overworked myself, without ensuring that the groups time is used effectively.

Task 2: Ensuring communication within the group

Problem & Solution 2.A: missing communication

There were a few times in the project in which I distributed tasks to people, that required them to contact one another to get information on how to collaborate (e.g. card design needs UX from Frieda, but also visuals from Oliver). However, communication between team members was sparse. As a result I had to arrange and partake in meetings which were about topics out of my expertise in order to ensure others have the relevant information for their task and know when to hand it over to another person.

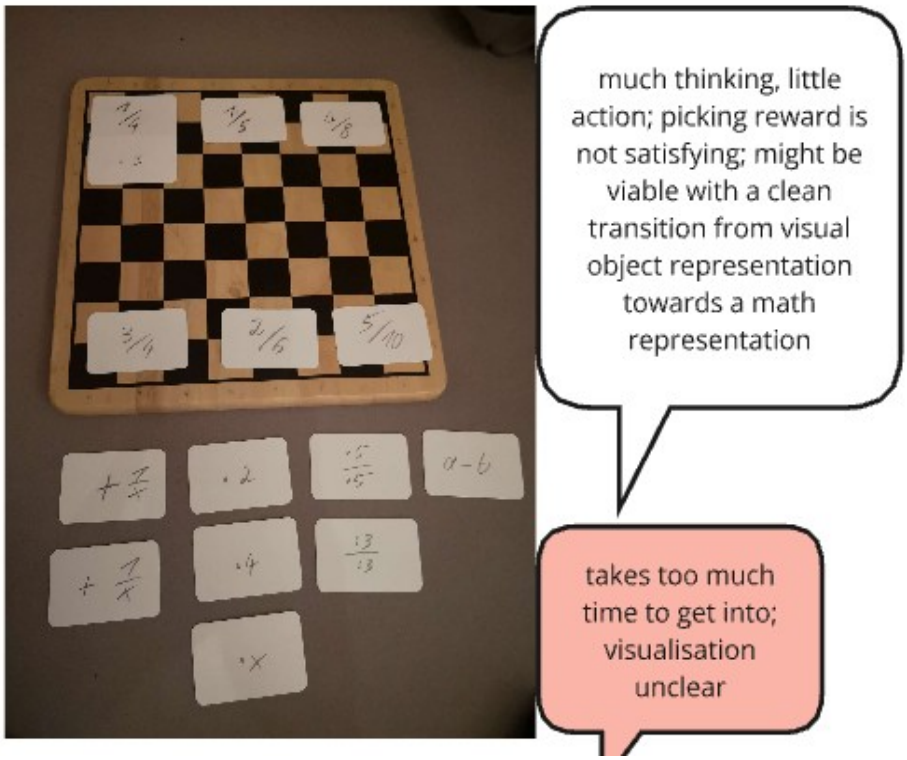
Unsolved Problem 2.B: communication problems

During the project there were multiple times, where I tried to bring some point across which didn't go well, due to any of the following reason:

- It turned into a heated debate
- My argument got lost somewhere along the communication
- I gave up on getting my point across

Example 2.B:

At the start of the project I did paper prototypes to test some game ideas. (see image with notes on the right side.)



Among them the deck-builder game which we chose. I warned my group that this form of gameplay consists of too much thinking with too little action / experimentation; Thus not being ideal as a teaching game.

They counterargued that the repeatability and thought process during gameplay helps to understand math better; Simultaneously game ideas / prototypes that encourage more experimentation were dismissed, because when players can just experiment wildly, it would discourage them from really getting into the math.

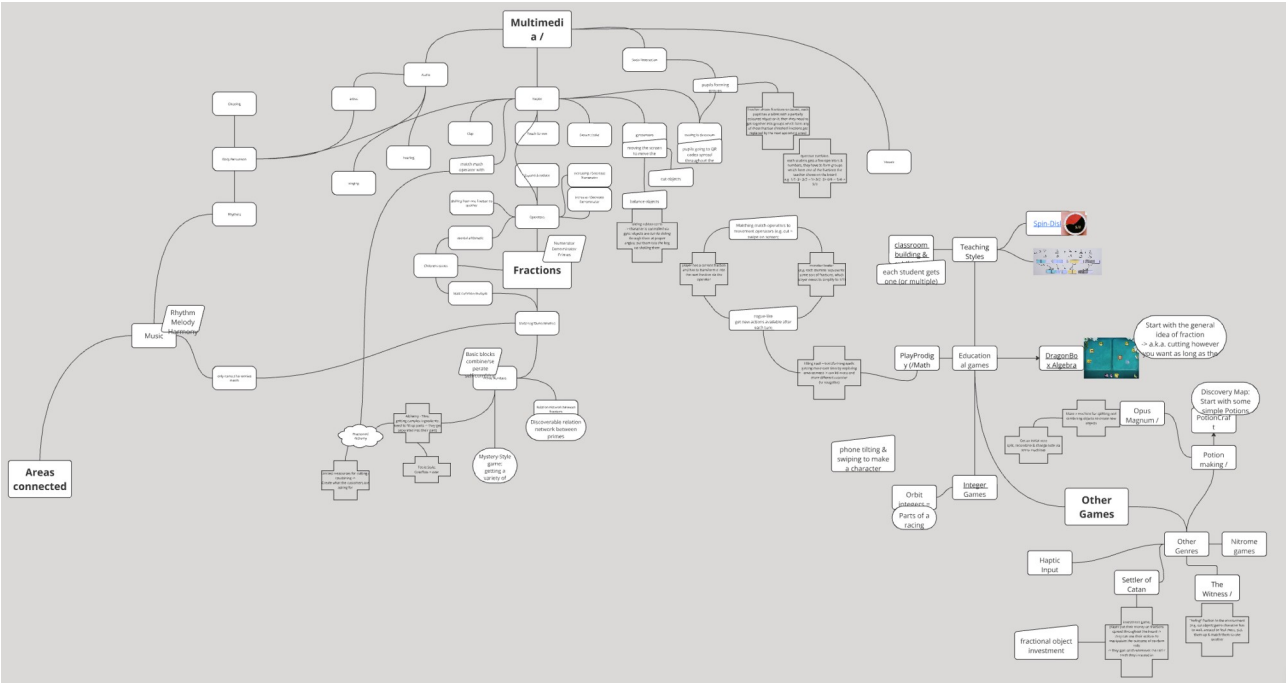
In my opinion the problems of our final game are precisely that it is too complicated to understand and the thinking process requires players to already understand math, instead of developing and understanding via experimentation or get better via actions in quick succession; Sadly it seems that even though I anticipated these problems very early, I wasn't capable of communicating these in a way that ensured we work on these problems.

Game Design

Task 3: Game Ideas

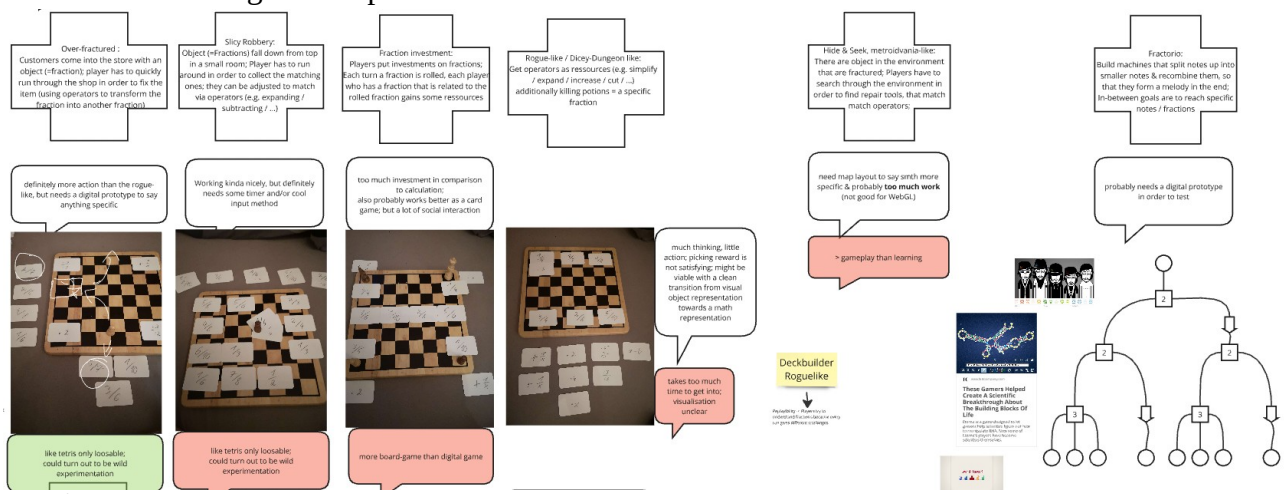
Step 3.A: brainstorming game ideas

On the first friday we did a designer brainstorm, which sadly didn't result in many useful ideas. Therefore over the weekend I created a bigger brainstorming mindmap, including inspiration from other games/genres.



(extract from my Brainstorming)

As a followup I paper-prototyped and playtesed a variety of the ideas for which I could come up with some form of game-loop.



(extract from paper-prototype images & notes about the testing)

Step 3.B: narrowing down ideas

We narrowed down the brainstormed ideas in two steps:

- An online designer meeting on monday morning:
  This was a bit exhausting to me, because the playtests gave me a rough idea how the games would feel to play; However the argumentation was rather theoretical & hypothetical and not based on testing (/my testing results), which made it hard for me to bring my points across.
  → 3 ideas left afterwards
- A full-group meeting monday afternoon:
  I suggested that we don't decide purely on how much we'd like the resulting game, but also based on a table that lists who'd have to do which tasks in the development process.
  → 1 main-idea left (and a less likely alternative option)

Step 3.C: client requirement documentation

We had a conversation with our client and Melina; Melina gave us her document for an educational game development process. We adjusted it to our needs and sent it to Eric. He filled it out by the next day, so that we knew the most relevant requirements:

- target audience: 5-6th graders
- topic: fractions
- setting: classroom

Step 3.D: first playtest

On Wednesday before the presentation, Frieda brought some ideas how the in-round combat for a deckbuilder could work. We tested them with playing cards to create our final game concept.

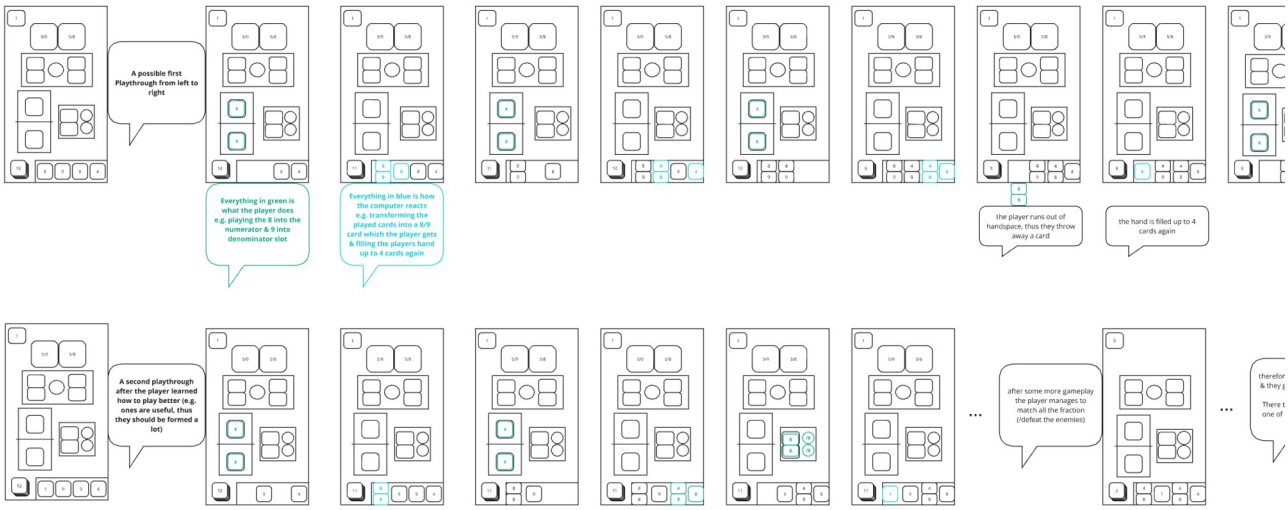
Task 4: Gameplay Documentation / Communication

Problem 4.A: documenting and communicating game concepts

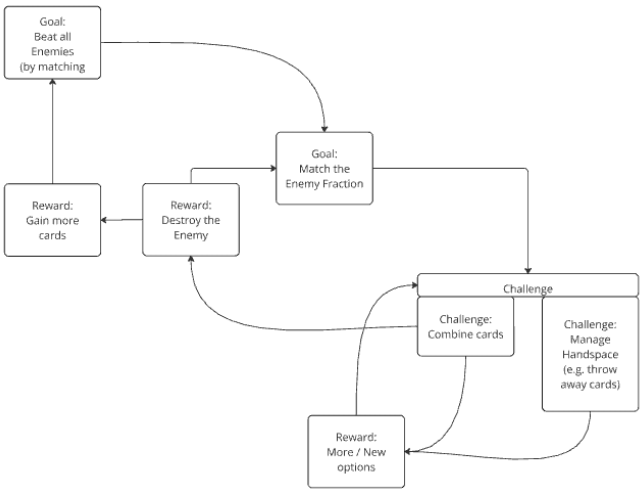
Throughout the development process there were multiple times, where we had to communicate our concept to the client or to other group members, so that they can work on it.

Solution 4.A: simple visualisers with explanatory text

I created a variety of visualisation with explanatory text to ensure that we fully agree on how something should work, before artists and programmers start implementing (based on the documents); A few examples include:

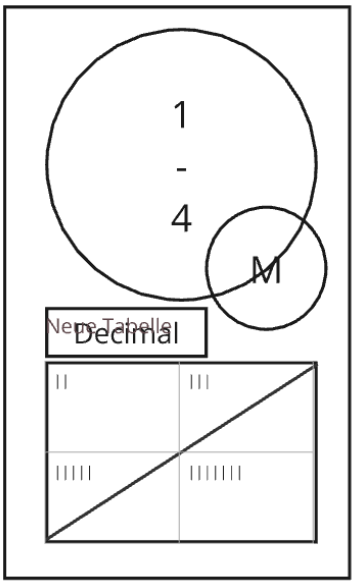


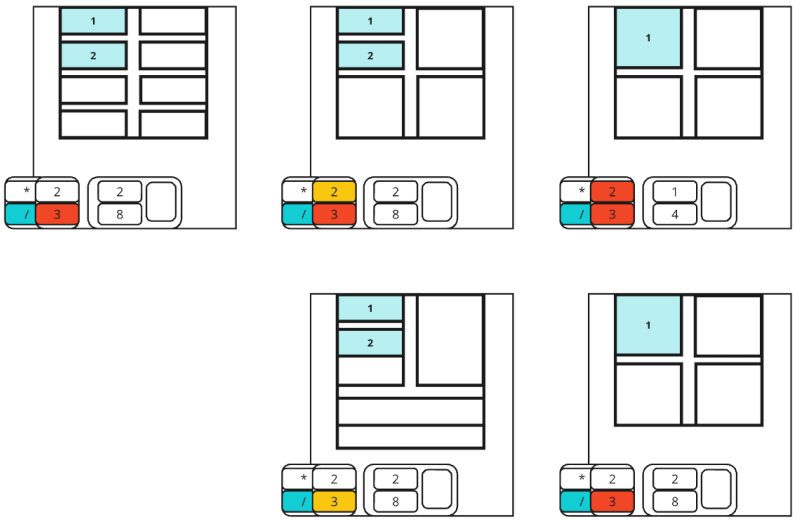
An initial gameplay example, which also convinced Eric of our gameplay idea.

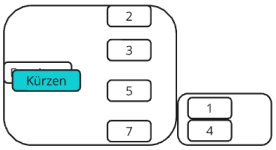


The gameloop

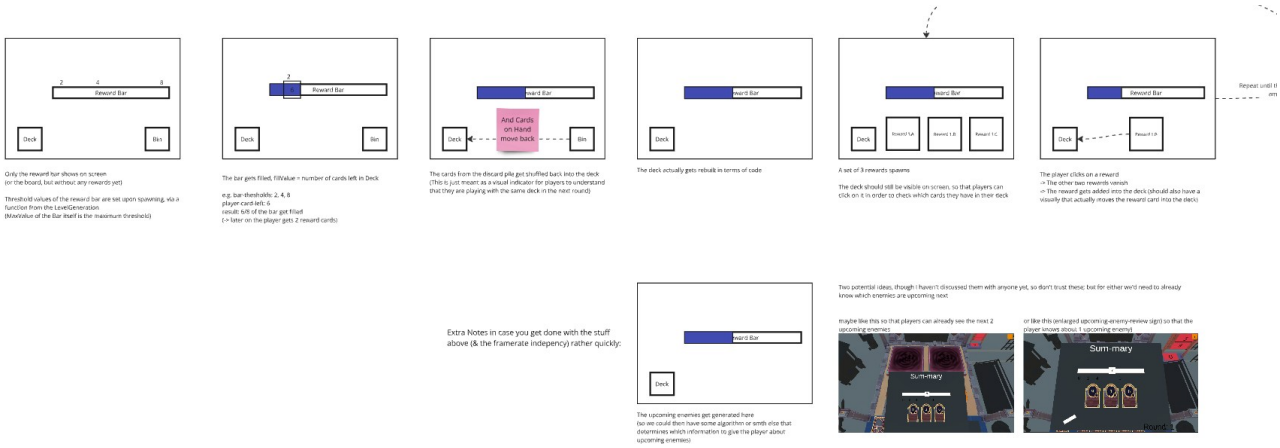
Information on the card, so that Oliver can create a new visualisation







Documentation how the expand & simplify menu should work via a pop-up.



Documentation how the player gets rewarded after each round (and refinement after the first reward-board was implemented)

Task 5: Playtests

Unsolved Problem 5.A: too detailed implementation before testing:

There were a multitude of features, that we already implemented with details before testing them properly. For example, I told Ahsan about how the reward-board slider should be animated, before the functionality itself got implemented.

Unsolved Problem 5.B: not caring about playtest results in time:

We did the first playtests of our game around the time of the intermediate presentation.
As these playtests were rather spontaneous, not everybody was there to watch them, so that we had to discuss what happened in them afterwards.
However, these playtests only lead to minor changes in our planned development process; We mostly continued working on what we had planned in the hope that our planned changes would solve the problems found in playtests.

Unsolved Problem 5.C: clashing opinions about playtesting results

In the sixth and seventh project week we did more playtests. Sadly at the second playtests I was the only one who actually saw people play the game who had nothing to do with it beforehand. That by itself was not the problem, as I made a video-recording of the playtest.
However after the playtests from the intermediate presentation and after the playtests in the last week we had very heated debates about which changes are needed. From my personal perspective, I saw that the game was way too hard to understand and too hard to play; From my perception the other designers argued on a more theoretical basis in favour of their UX- and level-design choices. We managed to agree on some changes to the game. Nevertheless, I believe that we have not changed enough, as the same points of criticism from the playtests appeared after the final presentation.

Learning outcome 5.A-C: more playtests

For my next project I've already planned to increase the amount of playtests to every week.
(bi-weekly for gameplay mechanics and bugs and bi-weekly for environment and level testing).
Additionally I'm using timeslots after each playtest, which exist purely to evaluate playtest data and restructure the project plan, before working on any further feature.

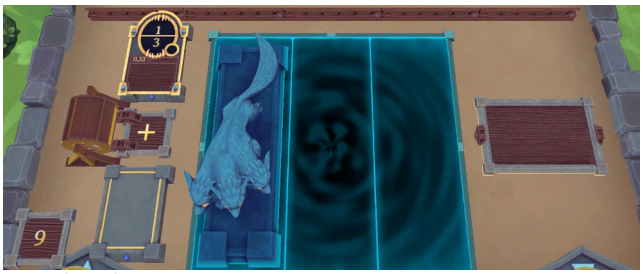
Programming

Task 6 figure visualisation on the board:

Problem 6.A: figure visualisation overengineering

Whenever the players drags a fraction *'x over y'* onto the board (or slots), the board has to react by spawning *x* figures on *y* tiles. Additionally, when they players wants to add or subtract fractions, the figures of the second fraction have to be in the correct addition / subtraction spots.

Another feature that I added, is that upon expanding simplifying figures, they still keep the same space on the board (instead of shifting around).



This shifting of the board-space taken is mostly avoided

Figures take up mostly the same space on the board, e.g. the top half in this case

On top of that I tried to write the code very expandable (e.g. ensure that more than 2 Operands can be visualised by only changing a few values and one code line per extra figure).
This quickly led to very complicated code, so that I gave up on it.

Solution & Learning outcome 6.A: only directly relevant functionality

I scaled down on writing code which already has the expanded features, so that I had more time to concentrate on making two operands with one operation work (even for bigger denominators).

In future projects I will ensure there is a detailed list of features that I need to implement at that specific point of time and implement exactly that.
If, at a later point, the feature needs to be expanded, then I can still refactor the code, but with an already working base. This removes time requirements and ensures something working can be tested before implementing extra features.

Task 7: Collaborating on Code

Problem 7.A: collaborative code chaos

At some points during the project we ran into the problem that one person needs to access functions in the script of another person.
Example: Whenever a card (scripts from Ahsan) gets put into one of the slots (scripts from Ahsan), the board needs to get notified to update the visualisation (my script).

Bad Solution 7.A: Making changes in other people code

For the given example, I went into Ahsan code and added a variety of events:

- OperationBoard.onChange-event
  - triggers Visualisation update
  - (and some other functionality)
- OperandSlot.onCardChange-event
  - triggers OperationBoard.onChange-event
  - subscribes to the cards onValueChange-event
- Cards.onValueChange-event
  - triggers cardVisualiser to update
  - triggers OperandSlot.onCardChange (for the subscribed Operand-Slot)

While this event-system itself worked, the approach of changing the scripts of one another (mostly me adjusting other scripts), lead to us sometimes loosing the overview over the code.

Good Solution / Learning outcome 7.B: Placeholder functions & events

There were features which we knew get implemented at some point;
For these we used some meeting to put empty functions into the correct place in the code.
With this method the code structure is done, so that nobody has to wait for one another, but can still take the required time to implement features.
Yet we didn’t do that often enough; especially for event structure & invocation I want to try the following workflow in upcoming projects (& should have used in this project):

1. Ahsan or me: Create a flowchart of events
2. Together: Discuss the flowchart & put the events in the code (so that other scripts can already work with them)
3. Each one individually: Implement event invokations into functions & subscribe listeners.

Something similar should help for class-hierarchy & object composition in order to avoid a lot of GetComponents, that we currently require as our cards have 6 different card-related components.

Learning outcome 7.C: more syntax knowledge

One advantage that came from looking into Ahsan's code, is that I learned other ways to structure code, especially in terms of syntax. Some useful features that I learned to use more include:

- Switch Expressions
- Local Functions
- #regions

Simplified Example:

```
private FractionVisualisationData VisualiseSingleFraction(OperandType opType, Fraction fraction)
{
    if (fraction is null) {throw new NotSupportedException();}
    [...]

    Operation operationCopy = CalcOperationCopy(); //see private function below
    Fraction combinedFraction =
            Fraction.CalculateOperation(leftOperand, operationCopy, rightOperand);
    Fraction visualisedFraction = CalcVisualisedFraction();
    List<Vector3Int> visualisedCoordinates = CalcVisualisedCoordinates();
    [...]

    FractionVisualisationData visData =
            new FractionVisualisationData(
            visualisedFraction, visStyle, figureOffsetAndSpacing, visualisedCoordinates);

     SwitchBoardVisualisationMode();

    return visData;



    Operation CalcOperationCopy()
    {
        return ((_operation == Operation.Add || _operation == Operation.Subtract) &&
        leftVisualisationData?.Item1.Denominator != rightVisualisationData?.Item1.Denominator ||
        (_operation == Operation.Subtract && rightData?.Item1 > leftData?.Item1)) ?
        Operation.Nop :
        _operation;
    }

    CalcVisualisedFraction()
    {
        ...
    }

    List<Vector3Int> visualisedCoordinates()
    {
        …
    }
}
```